

VCF-RDFizer: coauthor discussion brief

1 October 2026 • Current manuscript and next decisions

1. What is the paper's argument?

VCF-RDFizer turns genomic variant files into a shared graph with explicit links and access rules. These benefits must justify the cost of conversion.

VCF is a file format for genomic variants. RDF represents data as a graph of statements called *triples*; SPARQL is its query language. Conventional VCF tools can answer the same biological questions. The proposed benefit is making links between datasets and access rules explicit and independently checkable.

Meeting goal: agree on the central claim, the experiments needed before submission, and what belongs in the main text.

2. What has been demonstrated?

Which participants carry a ClinVar-classified variant in the 81 genes on the ACMG secondary-findings list, and which results may each requester receive?

ClinVar supplies variant classifications. We use real public genomes with **simulated consents**: research, clinical care only, or withdrawal. A further rule restricts results in 28 cancer-predisposition genes to clinical care. A clinical laboratory, cardiovascular study and biobank therefore receive different results.

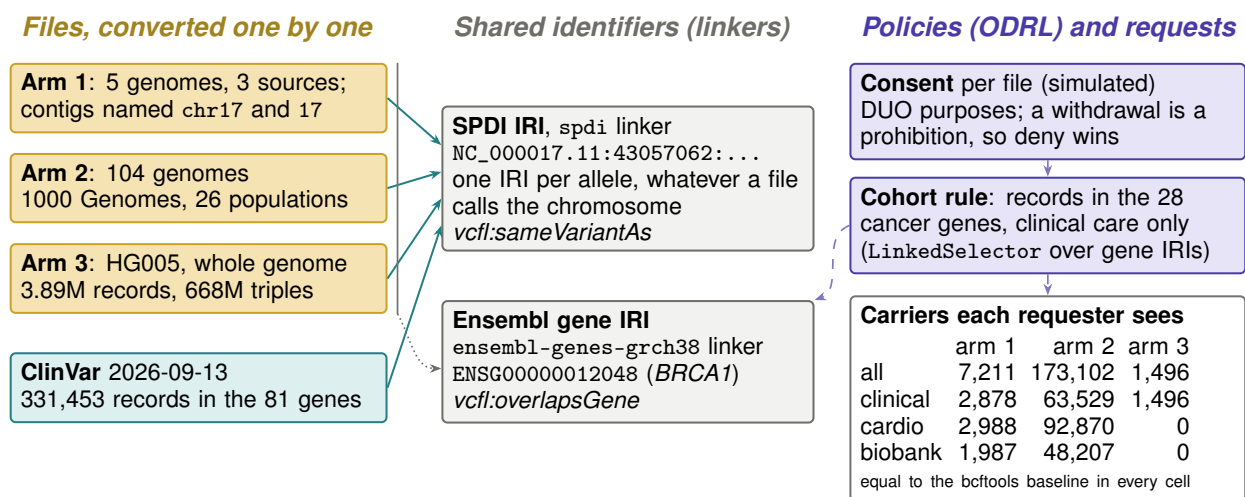
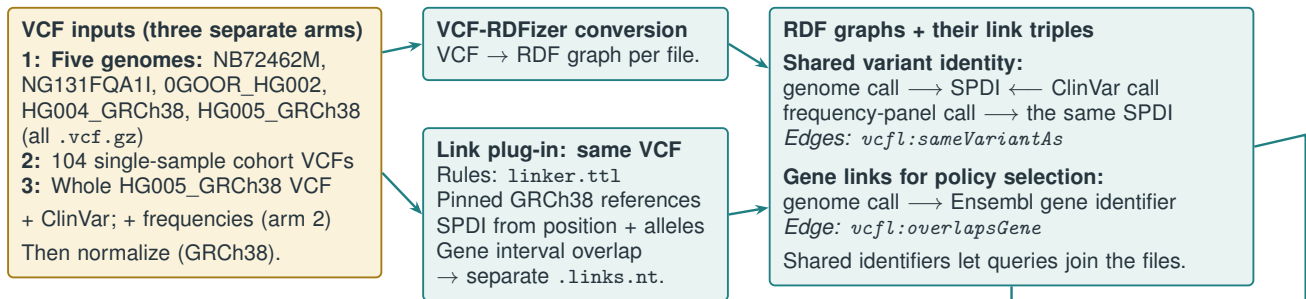


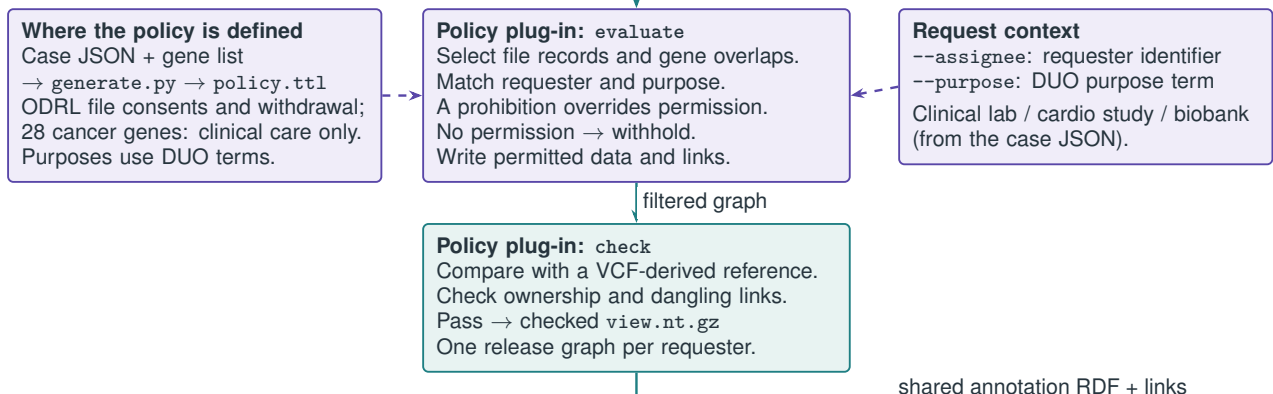
Figure 1. From genomes to permitted results. Read left to right: three dataset setups; shared variant and gene identifiers; access rules and results. An IRI is a shared identifier, SPDI identifies a variant, DUO names permitted purposes, and ODRL expresses the rules. Counts are participant-variant-gene matches, not numbers of people.

How the linking and policy framework works

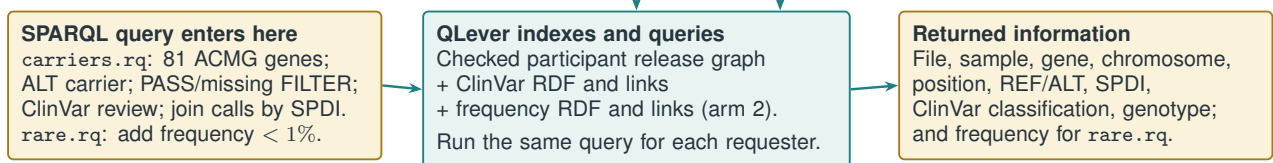
1. Prepare each file and add explicit links



2. Build a checked release



3. Query the permitted graph



Policy enforcement happens when the release graph is built. Requester identity and purpose are evaluation inputs; the biological query contains the analysis filters.

Figure 2. The implemented path from VCF files to a permitted answer. The diagram follows the use-case runner, policy generator, linker definitions and SPARQL files. Linkers read the normalized VCFs directly; conversion and links are stored separately. Inputs cover the gene regions except for whole HG005. The cancer-gene policy uses Ensembl overlap links, while the query reports the ACMG genes named in ClinVar's annotation.

What the experiment establishes

- **Every requester receives the same answer in both implementations:** the RDF route and an independent bcftools + Python pipeline. Each released dataset is also checked against the rules and the source VCF.
- Shared identifiers join variants despite different chromosome names; the cohort also joins a separate frequency file to identify rare variants.
- Results include *all* ClinVar classifications. The pathogenic/likely pathogenic subset has 0 carrier matches in the five-genome setup and 8 in the cohort.

Interpretation: this supports linked analysis and correct implementation of the simulated access rules. It does not establish real-world consent validity.

Validation: what was tested

3. How well is the conversion checked?

The test is whether conversion preserves the source data, and whether our checks would notice an error. It does not assess whether the original biological variant calls are correct.

The base benchmark contains 143 recorded cases across ten public files and controlled experiments that vary record or sample counts. The validation evidence comes from three complementary checks:

A. Do the VCF and RDF give the same answers?

We ask 13 questions twice: once by reading the original VCF, and once by querying the converted graph. These *paired comparisons* cover record and allele counts, genotypes, file metadata, and summaries of record and annotation values. They test more than whether conversion produced a readable file.

Recorded result	What was covered
984 matching answers; 0 disagreements	Fixtures and slices up to 100,000 records (17.1M triples).
4 comparisons not applicable	Two genotype questions on each of two empty-file artifacts; there are no records to compare.
1,144 consistency checks passed	Relationships between results, such as component counts summing to the expected total.

Separate retrieval experiments also found matching query answers on the 657M-triple whole-genome graph. These runs check those answers; they do not run the full set of structural checks described below.

B. Would a conversion error be detected?

Passing on valid data is only useful if the checks can fail on bad data. We therefore introduced 113 deliberate faults into test graphs, such as a wrong allele, coordinate, sample index or phasing value.

The query comparisons detected 96 of 113 faults (85%). The remaining 17 escaped because the queries do not inspect every relationship or value. SHACL adds rules about the graph itself: which values and links must exist, their datatypes, and whether related values agree. With all three SHACL rule sets (*profiles*) enabled, **113 of 113 faults were detected**; the two unchanged control graphs still passed.

The open experiment concerns the default settings. The full rule set is costly: on a roughly 2,000-triple graph, the two advanced profiles took 33.7 and 73.2 seconds, versus 1.7 seconds for the default profile. Routine validation therefore uses the default, with the full set reserved for small inputs. The default found no violations in 58 ordinary validations, but its detection rate on the 113 faulty graphs is not yet reported. These are different questions.

C. Can the compressed output be read back?

HDT and COTTAS are compressed RDF formats. All **152 artifacts** (118 HDT, 34 COTTAS) decoded successfully with the same triple count as their source graph. This covers larger outputs too, but matching counts alone cannot show that every value is preserved. In a separate round-trip test, compression and decompression also reproduced the original content digest.

Retrieval: the workload determines the benefit

4. When is the extra cost worthwhile?

Repeated questions can repay the conversion cost; a single batch of summaries does not in this experiment. We tested the same 13 questions on a 100,000-record HG005 slice (17.1M triples), comparing a direct VCF reader (`cyvcf2`) with QLever, a graph query engine. Answers agreed before timings were compared; results below average three repetitions.

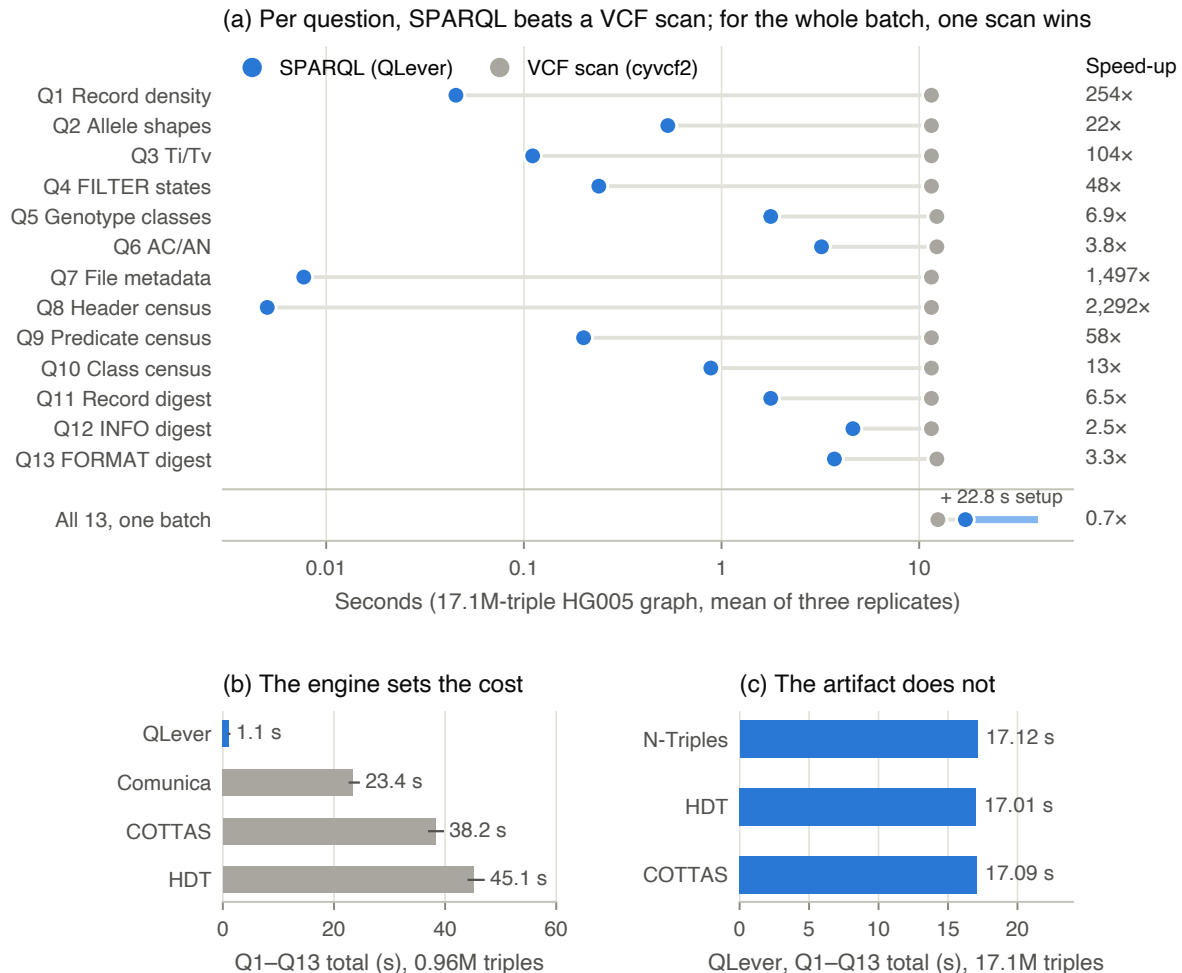


Figure 3. Query speed depends on the workload and engine. (a) Individual questions and the complete batch on the 17.1M-triple graph; conversion is excluded, and indexing appears only on the batch row. (b) Four engines on a smaller, 0.96M-triple graph, excluding setup. (c) QLever after loading the same 17.1M-triple graph from three storage formats. Panels (b) and (c) use different graph sizes.

Read the speed-up together with its setup cost

- **One question:** a VCF scan takes 11.5 s; an indexed QLever query takes 0.005–4.62 s. However, conversion takes 423.7 s and QLever indexing a further 22.8 s before the first query.
- **All 13 questions together:** one VCF pass takes 12.4 s, versus 17.1 s of QLever query time, even before setup. Batching avoids repeated VCF scans.
- **Repeated, separate questions:** QLever averages 1.32 s per question, saving about 10.2 s per VCF scan. Recovering the roughly 447 s conversion and indexing cost takes **about 44 questions**, assuming this question mix and reuse of the same graph. This is not a universal threshold.

QLever builds its own index from each format. Query times differ by under 5% (Figure 3c); HDT and COTTAS chiefly affect storage and transfer in this test.

Cost trade-offs: region size and sample representation

Indexed VCF is a stronger baseline for regional lookups

Tabix indexes VCF records by genomic position, avoiding a whole-file scan. We compared five regional questions over 20 windows per size, with three repetitions. Across both tested graphs, 11,160 executions had no failures or disagreements. On the 17.1M-triple graph:

Window width	QLever	Indexed cyvcf2	Indexed bcftools
1 kb (1,000 bases)	9.6 ms	3.3 ms	4.3 ms
10 Mb (10 million bases)	11.1 ms	117.8 ms	32.7 ms

Median query times. Setup is excluded: QLever indexing took 23.2 s; VCF compression and tabix indexing took 0.39 s. Narrow windows favour indexed VCF; QLever wins for the broad windows tested.

Sample profiles change the amount of graph we need to build

We held 10,000 variant records fixed and increased the sample columns from 1 to 2,504. **Expanded** stores each sample call as graph resources and values. **Condensed** packs values into sample-ordered vectors: fewer triples, but retrieving an individual sample value requires decoding its position.

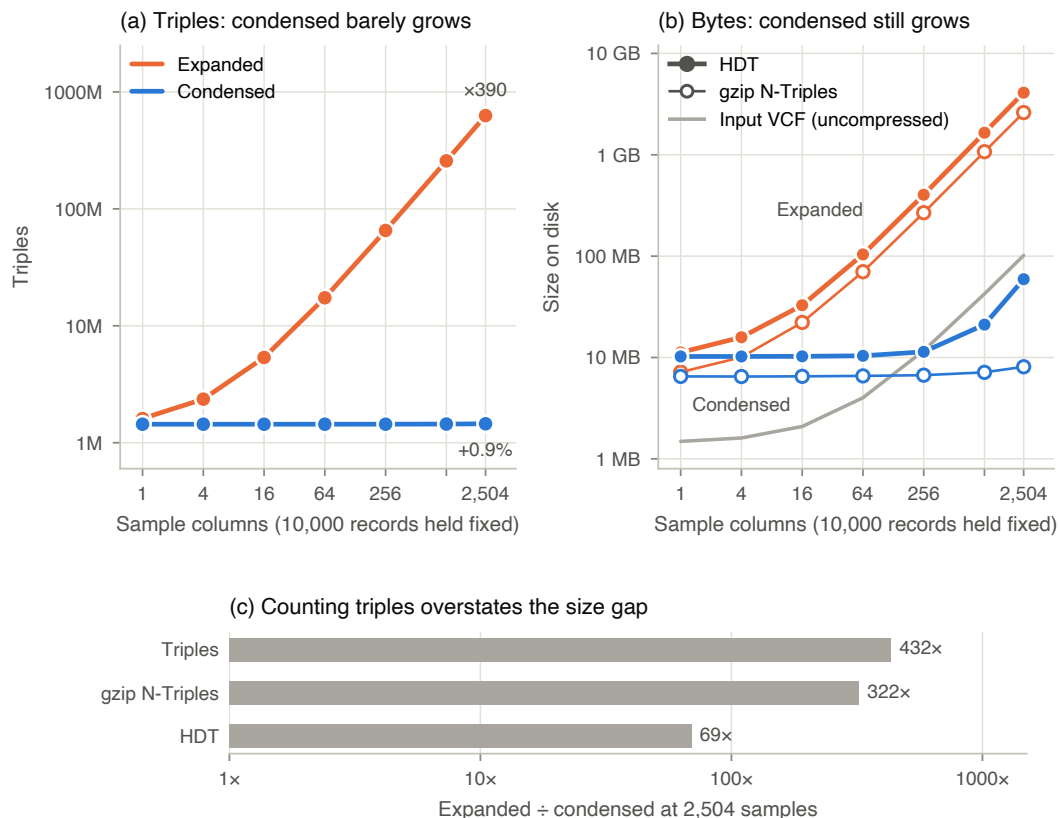


Figure 4. Fewer triples do not mean that sample data disappear. Condensed vectors lengthen as samples are added: their HDT grows from 10.2 to 59.2 MB. At 2,504 samples, expanded has 432 times the triples but 69 times the HDT bytes (4.10 GB). The $\times 390$ label in (a) instead measures expanded growth from 1 to 2,504 samples.

Storage and construction remain substantial costs

In the whole-genome benchmark, producing 657M triples and compressed outputs took **16.1 hours**, including 14.5 hours building HDT and COTTAS. HDT, its index and COTTAS together occupied 10.88 GB versus a 139 MB compressed VCF download (78 times larger). This is a different configuration from the clinical use case's 53-minute conversion, which reports later stages separately.

What should we address before submission?

5. Current state and immediate work

The draft contains the use case, performance comparisons and revised manuscript structure. It has about 23 narrative pages (32 including declarations and references), plus a 9-page supplement. Two targeted experiments remain unreported in the current draft:

Experiment	Next action	Result needed
Default validation	Run the 113 injected faults with only the default SHACL profile.	Faults detected, types missed and runtime under normal settings.
Different input source	Run full validation on a 250,000-record genome slice from another sequencing provider.	Paired query agreement and structural-check results beyond the existing reference slice.

Before submission: complete funding, archive identifiers and release metadata; confirm the final release and automated-test status. The draft assigns v3.1.0 to the base benchmark and v3.2.0 conversion/v3.3.0 plug-ins to the use case; final results and version identifiers must correspond.

6. Decisions for the coauthors

1. **Is the demonstrated benefit convincing?** Are shared identifiers, inspectable policies and checked release sufficient with simulated consents, or do we need an external user or data-access partner? Rules take 72 authored lines in RDF versus 131 in the baseline, but three of four tested changes are similarly small in both. This does not establish a general usability advantage.
2. **How much validation is enough?** Are the two targeted runs above sufficient with the reported scope, or is complete whole-genome validation required? The latter needs more work on the costly structural checks.
3. **Do we need a measured converter comparison?** The current comparison with other converters is based on documentation. Decide whether to add one shared-input experiment and which outcomes it should compare.
4. **What belongs in the main story?** Keep conversion, validation and the linked-data use case central; present HDT/COTTAS as measured storage choices. Agree whether the policy component belongs here or warrants a separate paper, and how this submission relates to the companion vocabulary paper.
5. **What can wait?** Federation with other graph services, alternative variant identifiers, whole-cohort conversion and sample-level access control in multi-sample files are future work unless essential to the central claim. Choose which detailed tables move to the supplement.

Reading map: manuscript Sections 4.1 (fidelity), 4.2 (use case), 4.3–4.4 (costs), and 5 (interpretation); Supplementary Sections S1–S5 (execution, coverage and testing issues). Experiment status follows `tool-docs/workstream-a-implementation.md` and the archived scale-retrieval reports. Figures 1, 3 and 4 are from the current manuscript; Figure 2 follows `benchmarks/17_use_case_acmg.sh`, its case files and the tool's linking and policy implementation.